



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-[Section] System Integration and Architecture

System Design Document (SDD)

Project Title: PocketBooks

Prepared By: Base, Jascha Billon

Version: 0.1

Date: 2/18/2026

Status: In Progress

REVISION HISTORY TABLE

Versio n	Date	Author	Changes Made	Status
0.1	2/18/2026	Jascha B. Base	Initial draft	Draft
0.2	[Date]	[Your Name]	Added API specifications	Review
0.3	[Date]	[Your Name]	Updated database design	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY.....	4
1.0 INTRODUCTION.....	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION.....	5
3.0 NON-FUNCTIONAL REQUIREMENTS.....	9
4.0 SYSTEM ARCHITECTURE.....	11
5.0 API CONTRACT & COMMUNICATION.....	12
6.0 DATABASE DESIGN.....	17
7.0 UI/UX DESIGN.....	18
8.0 PLAN.....	21

EXECUTIVE SUMMARY

1.1 Project Overview

PocketBooks is a personal finance management system tailored for students to track daily allowances, academic expenses, and savings goals. The system consists of a **Spring Boot** backend API and a **React** web application, providing a secure and organized way to manage student finances.

1.2 Objectives

1. Implement a secure authentication system featuring **JWT** and **Google OAuth**.
2. Develop a **Core Business Module** for managing academic expenses with full CRUD functionality.
3. Integrate third-party services including an **External Currency API**, **File Upload** for receipts, and a **Sandbox Payment Gateway**.
4. Follow a **Layered Architecture** pattern to ensure code maintainability and scalability.

1.3 Scope

Included Features:

- **Authentication:** User registration, Login (JWT), Google Social Login, and Logout.
- **Expense Management:** Full CRUD for student expenses with category tagging.
- **External Integration:** Real-time currency conversion via external API.
- **Document Handling:** Uploading and viewing digital copies of receipts (PDF/Images).
- **Payment Integration:** Sandbox "Allowance Top-up" feature to simulate adding funds.
- **Email System:** Automated welcome emails and transaction notifications.
- **RBAC:** Role-based access for Students (regular users) and Administrators.

Excluded Features:

- Real-world financial transactions (Live Mode).
- Multi-currency wallet holding (Display only).
- Advanced investment tracking or stock market integration.

1.0 INTRODUCTION

1.1 Purpose

This document provides the technical specifications and design for the **PocketBooks** system. It serves as a guide for the final implementation, detailing the functional requirements, system architecture, database schema, and security protocols required for the IT342 final defense.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: Marketplace MVP

Domain: E-commerce/Retail

Primary Users: Customers and Administrators

Problem Statement: Users need a simple, fast way to browse and purchase products without the complexity of large e-commerce platforms.

Solution: A minimalist marketplace focusing on core shopping functionality with consistent experience across web and mobile.

2.2 Core User Journeys

Journey 1: Account Setup & Social Login

1. Account Setup & Social Login
2. User visits the landing page.
3. Clicks "Sign in with Google."
4. System links Google account to the database and generates a JWT.
5. User is redirected to the Dashboard to set their initial allowance

Journey 2: Logging an Academic Expense with Receipt

1. User clicks "Add Expense."
2. Enters amount, category (e.g., "Books"), and description.
3. Uploads a photo of the receipt.
4. System saves the record, links the file, and updates the total balance.

Journey 3: Administrator Product Management

1. Admin logs in with administrative credentials.
2. Navigates to "Global Settings."
3. Adds or edits global expense categories available to all users.

Journey 4: Wallet "Top-up" via Sandbox Payment

1. User navigates to the Wallet section.
2. User selects an amount (e.g., ₱500) and clicks the PayPal Sandbox button.
3. A popup appears for the user to log in to their PayPal Sandbox account.
4. Upon successful payment, the Spring Boot backend receives a confirmation, updates the balance in the Users table, and logs the transaction.
5. The UI displays a "Payment Successful" message and updates the balance in real-time.

2.3 Feature List (MoSCoW)

MUST HAVE

1. **Authentication & Security:** User registration, Login, Logout, JWT implementation, and Password hashing (BCrypt).
2. **Core Business Module (Expenses):** Full CRUD operations for logging, viewing, editing, and deleting student expenses.
3. **Role-Based Access Control (RBAC):** Implementation of Student (User) and Admin roles with API and UI-level restrictions.
4. **Receipt Management:** File upload functionality for digital receipts (Images/PDFs) linked to specific records.
5. **Google OAuth:** Social login integration that links/saves users to the database.

SHOULD HAVE

1. **Expense Categorization:** System for tagging expenses (e.g., Food, Tuition, Transportation) with filtering capabilities.
2. **Currency Conversion:** Integration with an External API to view balances in multiple currencies.
3. **Payment Sandbox:** Integration of a test-mode payment gateway (PayPal/Stripe) for simulated allowance "top-ups."
4. **Email Notifications (SMTP):** Automated welcome emails and system-generated transaction receipts.

5. **Responsive Dashboard:** A UI that adapts to mobile and desktop screens for on-the-go tracking.

COULD HAVE

1. **Real-time Updates:** WebSocket or Polling integration for live balance and notification refreshes.
2. **Dark Mode UI:** Theme toggling for better user experience in different lighting.
3. **Data Visualization:** Basic charts (Pie/Bar) showing a breakdown of spending by category.

WON'T HAVE

1. **Live Bank Integration:** Real-time syncing with actual bank accounts or credit cards.
2. **Crypto Payments:** Support for digital currency transactions.
3. **Real-world Financial Processing:** Live payment mode (System will remain in Sandbox/Test mode).
4. **Multi-language Support:** The initial MVP will be English-only.

2.4 Detailed Feature Specifications

Feature: Security and Identity

- **Functions:** Registration, Login, Google OAuth, Password Hashing (BCrypt).
- **API Endpoints:** POST /api/auth/login, POST /api/auth/register, GET /api/auth/me.
- **Security:** JWT for token management, Protected Routes in React.
-

Feature: Core Expense Module

- **Functions:** Add, Edit, View, and Delete expense records.
- **Fields:** Amount, Date, CategoryID, Description, ReceiptURL.
- **API Endpoints:** GET /api/expenses, POST /api/expenses, DELETE /api/expenses/{id}.

Feature: External Integration (Currency & Files) Shopping Cart

- **External API:** Consumes **ExchangeRate-API** to show balance in different currencies.
- **File Upload:** Multi-part file upload for receipts stored on the server

2.5 Acceptance Criteria

AC-1: Successful User Registration & Security

Given I am a new user

When I enter valid email and strong password

And confirm password matches

And click "Create Account"

Then my account should be created

Then my account should be created in the database with a hashed password (BCrypt)

And I should receive a JWT and be redirected to the Dashboard

AC-2: Logging an Expense with Receipt Upload

Given I am logged in as a student

When I fill out the expense form with an amount and category

And I attach a valid file (JPG/PDF) under 2MB

And I click "Save Expense"

Then the record should be saved and linked to the file on the server

And my total balance should automatically decrease by the expense amount

And I should be able to view the receipt file from my history list

AC-3: Wallet Top-up via Payment Sandbox

Given I am on the "Add Funds" page

When I select an amount and complete the payment via the PayPal/Stripe Sandbox

And the payment provider returns a "Success" status

Then the system should record the transaction ID in the database

And my account balance should increase immediately

And I should receive a confirmation email via SMTP

AC-4: Currency Conversion Display

Given I have an existing balance in PHP

When I select a different currency (e.g., USD) from the dropdown

Then the system should fetch the latest rate from the External API

And the UI should display the converted balance accurately without refreshing the page

AC-5: Admin Role-Based Access (RBAC)

Given I am logged in with an "Admin" role

When I navigate to the "Category Management" section

Then I should be able to Create, Update, or Delete global expense categories

But When I am logged in as a "Regular User," I should be restricted from accessing this page

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API Response Time: ≤ 2 seconds for 95% of requests (including External API calls for currency).
- Currency Conversion Latency: External API data fetching and calculation should complete within 1.5 seconds.
- File Upload Speed: A 2MB receipt file should upload and process within 5 seconds on a standard 10Mbps connection.
- Concurrent Users: Support up to 50 concurrent student sessions without degradation of service.
- Database Efficiency: All CRUD queries (Expenses/Users) must complete within 500ms.

3.2 Security Requirements

- Data Encryption: HTTPS for all communications; no sensitive data in URLs.
- Authentication: JWT-based stateless authentication with a 24-hour expiration token.
- Password Security: Mandatory password hashing using BCrypt with a minimum strength of 10 salt rounds.
- File Security: Uploaded receipts must be renamed on the server to prevent directory traversal attacks.
- Access Control: RBAC (Role-Based Access Control) enforced at both the API level (Spring Security) and UI level (React Protected Routes).
- Sensitive Data: DTOs (Data Transfer Objects) must be used to ensure password_hash or internal IDs are not exposed in JSON responses.

3.3 Compatibility Requirements

- Web Browsers: Chrome, Firefox, and Safari (latest 2 versions).
- Screen Responsiveness: Must be fully functional on mobile (\$360px+\$) for on-the-go expense logging and desktop (\$1024px+\$) for detailed reports.
- Network: Must handle "Offline" states gracefully using React error boundaries if the External Currency API is unreachable.

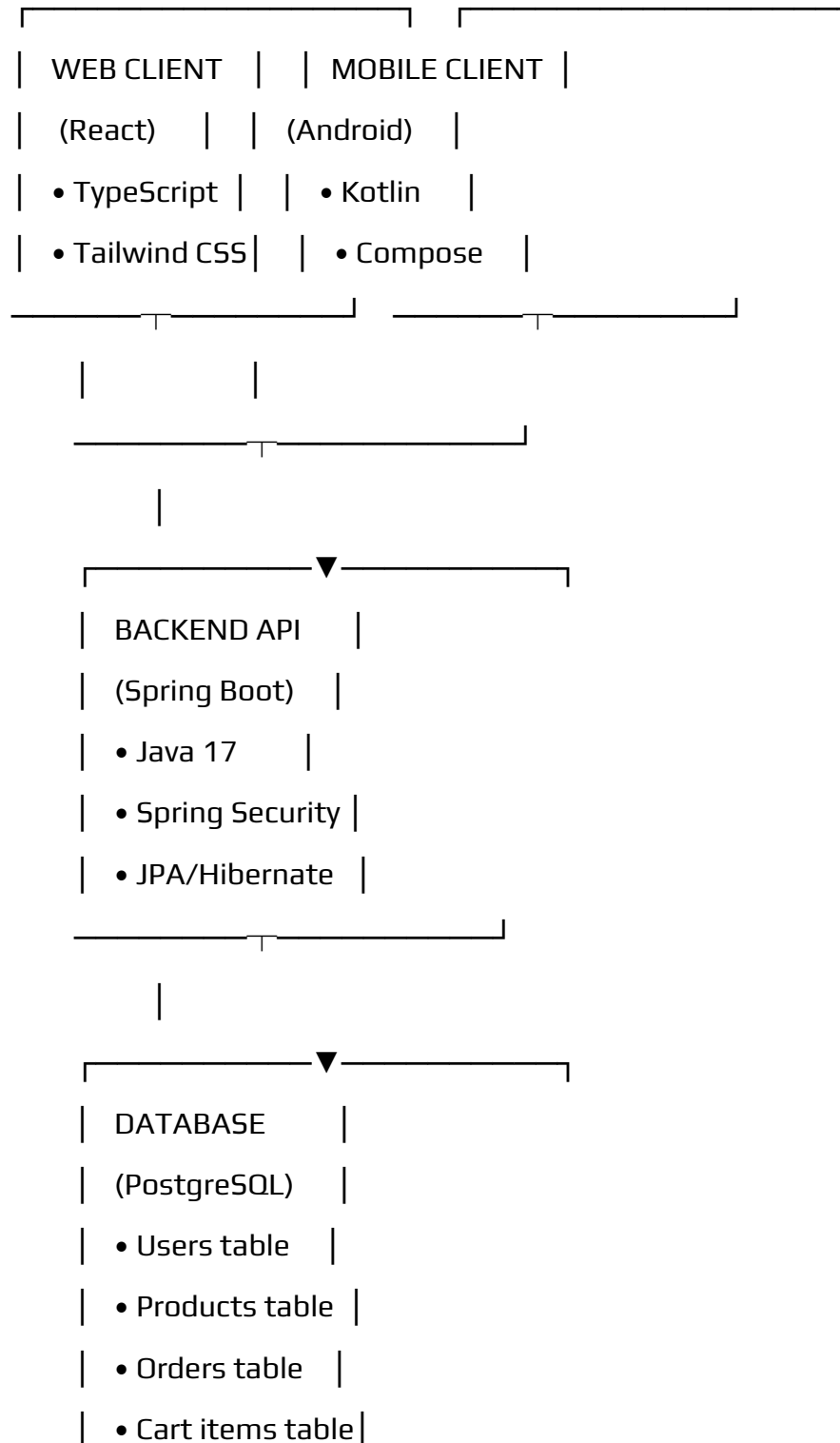
3.4 Usability Requirements

- Efficiency: A student should be able to log an expense and upload a receipt in under 30 seconds.
- Error Handling: Clear, user-friendly error messages for failed payments (Sandbox) or invalid file types (e.g., "Only JPG and PDF allowed").
- Navigation: A persistent sidebar or bottom navigation bar for quick access to the Dashboard, History, and Wallet.
- Visual Clarity: High contrast for financial figures (green for balance increases, red for expenses) to ensure readability.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, Jetpack Compose, Retrofit, Room
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

- **Base URL:** <https://api.marketplace.com/api/v1>
- **Format:** JSON for all requests/responses
- **Authentication:** Bearer token (JWT) in Authorization header
- **Response Structure:**

json

```
{  
  "success": boolean,  
  "data": object|null,  
  "error": {  
    "code": string,  
    "message": string,  
    "details": object|null  
  },  
  "timestamp": string  
}
```

5.2 Endpoint Specifications

Authentication Endpoints

POST /auth/register

Body: {email, password, confirmPassword, fullName?}

Response: {user: {id, email, name}, token, refreshToken}

POST /auth/login

Body: {email, password}

Response: {user: {id, email, name, role}, token, refreshToken}

POST /auth/logout

Headers: Authorization: Bearer {token}

Response: {message: "Logged out successfully"}

Product Endpoints

GET /products

Query: ?page=1&limit=20&search=keyword&category=electronics

Response: {products: [...], pagination: {page, limit, total, pages}}

GET /products/{id}

Response: {product: {id, name, description, price, stock, imageUrl, category}}

POST /products (Admin only)

Body: {name, description, price, stock, imageUrl, category}

Response: {product: {...}}

PUT /products/{id} (Admin only)

Body: {name?, description?, price?, stock?, imageUrl?, category?}

Response: {product: {...}}

Cart Endpoints

GET /cart (Authenticated)

Response: {cart: {id, items: [...], total, itemCount}}

POST /cart/items

Body: {productId, quantity}

Response: {message: "Added to cart", cartItem: {...}}

PUT /cart/items/{itemId}

Body: {quantity}

Response: {message: "Cart updated", cartItem: {...}}

DELETE /cart/items/{itemId}

Response: {message: "Removed from cart"}

Order Endpoints

POST /orders

Body: {shippingAddress: {fullName, address, city, zipCode, country}}

Response: {order: {id, orderNumber, total, status, items: [...], createdAt}}

GET /orders

Response: {orders: [...]}

GET /orders/{id}

Response: {order: {...}}

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

json

```
{  
  "success": false,  
  "data": null,  
  "error": {  
    "code": "AUTH-001",  
    "message": "Invalid credentials",  
    "details": "Email or password is incorrect"  
  },  
  "timestamp": "2024-01-28T10:30:00Z"  
}
```

```
{  
  "success": false,  
  "data": null,
```

```

"error": {
  "code": "VALID-001",
  "message": "Validation failed",
  "details": {
    "email": "Email is required",
    "password": "Must be at least 8 characters"
  }
},
"timestamp": "2024-01-28T10:30:00Z"
}

```

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

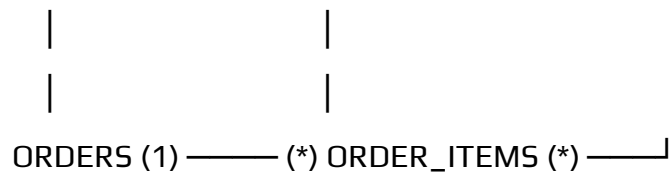
6.1 Entity Relationship Diagram

Note: This should be an ERD

```

USERS (1) — (1) CARTS (1) — (*) CART_ITEMS (*) — (1) PRODUCTS
|
|
|
(*)
|
|
|
(*)

```



Detailed Relationships:

- **One-to-One:** User ↔ Cart (Each user has exactly one cart)
- **One-to-Many:** User → Orders (User can have multiple orders)
- **One-to-Many:** Cart → CartItems (Cart contains multiple items)
- **One-to-Many:** Order → OrderItems (Order contains multiple items)
- **Many-to-One:** CartItems → Product (Items reference products)
- **Many-to-One:** OrderItems → Product (Items reference products)

Key Tables:

1. **users** - User accounts and authentication
2. **products** - Product catalog information
3. **carts** - Shopping cart per user
4. **cart_items** - Items in shopping cart
5. **orders** - Customer orders
6. **order_items** - Items in each order
7. **refresh_tokens** - JWT refresh tokens

Table Structure Summary:

- **users:** id, email, password_hash, full_name, role, created_at
- **products:** id, name, description, price, stock, image_url, category
- **carts:** id, user_id, created_at
- **cart_items:** id, cart_id, product_id, quantity
- **orders:** id, order_number, user_id, total, status, shipping_address
- **order_items:** id, order_id, product_id, product_name, quantity, price

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma

Homepage (Product Listing)

Header: [Logo] [Search Bar] [Cart Icon] [User Menu]

Content: Product Grid (3 columns desktop)

Each Product Card: Image, Name, Price, "Add to Cart" button

Footer: Links, Copyright

Product Detail Page

Back Button

Product Image (large)

Product Name and Price

Description

Quantity Selector (1-10)

"Add to Cart" and "Buy Now" buttons

Product Specifications

Shopping Cart Page

Cart Title

List of Cart Items (Image, Name, Quantity, Price, Remove)

Order Summary: Subtotal, Shipping, Tax, Total

"Continue Shopping" and "Proceed to Checkout" buttons

Checkout Page

Shipping Address Form

Order Review (Items, Prices, Totals)

"Place Order" button

Terms and Conditions note

Admin Dashboard

Sidebar Navigation: Dashboard, Products, Orders, Users

Product Management: Add New button, Product list with Edit/Delete

Order Management: Order list with status filters

7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma

Bottom Navigation

[ Home] [ Search] [ Cart] [ Profile]

Home Screen

Search Bar

Product Grid (2 columns)

Swipe gestures for quick actions

Pull to refresh

Product Detail Screen

Back arrow

Product image (swipeable gallery)

Product info

Quantity selector

"Add to Cart" fixed bottom button

Cart Screen

Edit mode for quantity updates

Swipe to remove items

Order summary sticky bottom

Checkout button

Checkout Flow

Step indicator: Cart → Shipping → Payment → Confirm

Address form (auto-complete)

Order summary

Place order button

Mobile-Specific Features:

- Touch-optimized buttons (min 44x44px)
- Gesture support (swipe, pull-to-refresh)
- Offline caching for product images
- Bottom navigation for main actions
- Simplified forms for mobile input

Design System:

- **Colors:** Primary (#2563EB), Secondary (#7C3AED), Success (#10B981), Error (#EF4444)
- **Typography:** Inter font family, responsive sizing
- **Spacing:** 8px grid system
- **Components:** Consistent buttons, inputs, cards, modals
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024px

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with dependencies

Day 2: Database configuration and entities

Day 3: JWT authentication implementation

Day 4: User management endpoints

Day 5: Product CRUD operations

Week 4: Core Features

Day 1: Cart functionality

Day 2: Order management

Day 3: Search and filtering

Day 4: Error handling and validation

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup with TypeScript
Day 2: Authentication pages (login, register)
Day 3: Product listing page
Day 4: Product detail page
Day 5: Shopping cart implementation

Week 6: Complete Web Features

Day 1: Checkout flow
Day 2: Order history and confirmation
Day 3: Admin dashboard
Day 4: Responsive design polish
Day 5: API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure
Day 2: Authentication screens
Day 3: Product browsing
Day 4: Shopping cart
Day 5: API service layer

Week 8: Complete Mobile App

Day 1: Checkout flow
Day 2: Order management
Day 3: UI polish and animations
Day 4: Testing on emulator/device
Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

- Day 1: End-to-end testing across platforms
- Day 2: Bug fixes and optimization
- Day 3: Security review
- Day 4: Performance testing
- Day 5: Documentation updates

Week 10: Deployment

- Day 1: Backend deployment (Railway/Heroku)
- Day 2: Web app deployment (Vercel/Netlify)
- Day 3: Mobile APK distribution
- Day 4: Final testing
- Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Product catalog API (Week 3-4)
3. Shopping cart functionality (Week 4)
4. Checkout process (Week 6)
5. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements